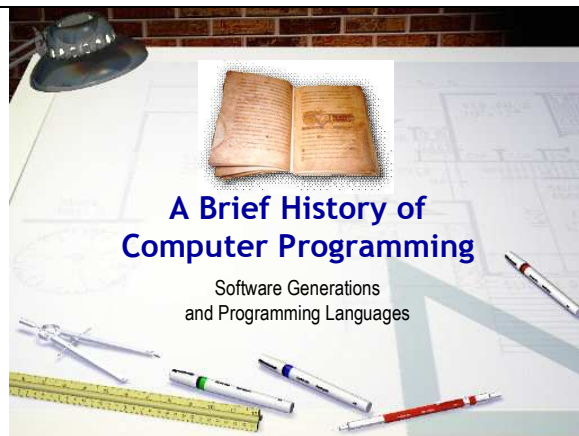




Slide 1

A Brief History of Computer Programming

Duration: 00:01:25

Advance mode: Auto

Notes:

Before we dive into the Java programming language itself, let's take a look at where Java came from, on the theory that George Santayana was right when he wrote, "Those who ignore history are doomed to repeat it."

It's almost impossible to understand why Java works the way it does without understanding a little bit about other computer programming languages—both modern and "ancient"—and how and why they were developed. In this lecture, you'll learn a little about the history of computers, and computer programming.

As you saw in previous lectures, modern computer systems (in general) consist of at least four pieces:

- A processing unit (CPU) that performs simple arithmetic and comparisons.
- Memory that can store information: both data and instructions used by the processing unit.
- Input/Output devices that allow you to load information into memory and to see the results of your calculations.
- Storage devices that allow you to keep your calculations and your programs for later reuse.

The instructions that tell the processing unit which calculations to perform, and which information to use in those calculations, is called a program. Programming is the art (or science, or even black-magic, if you prefer) of creating programs.

Let's see what programs are, and where programming came from.



Slide 2

Slide 2

Duration: 00:03:25

Advance mode: Auto

Notes:

Here's something most of you are familiar with. This is a recording of Dick Dale and His Del-Tones greatest hits from 1961 to 1976. This tune is called "The Wedge". When I was young, I'd ride my bike down to Balboa and listen to Dick and the Del-Tones playing the Rendezvous Ballroom, and, if I wanted to listen to the music at home, I'd purchase a recording that looked a lot more like this—the analog audio signals from Dale's guitar were etched into a flat vinyl disc and were played back by amplifying the sound produced by tracing the grooves with a metal phonographic stylus. Of course, each time you spun the disc, the sound changed slightly as microscopic portions of the recording were scraped away. Audio aficionados though, still swear by these gramophone records, since the spiral groove cut into the record exactly reflects the analog waveform of the original sound.

Today, of course, vinyl recordings are a thing of the past. Instead, the Compact Disc shown here uses a digital recording technology, where the original sound way is first sampled, and then reduced to a series of 1s and 0s, just like a

computer program. These 1s and 0s are then etched into a plastic platter using a laser to create a series of microscopic raised and lowered areas, called lands and pits. Once the CD is burned, you can listen to the music by using a CD player, like my daughter's Walkman, which uses its own laser to read the microscopic binary signals, and then reconstruct the original sound wave recorded on the disc. Once the sound is decoded, the analog output is sent to your speakers or headset.

This is similar to the way your computer works. Like your computer, the CD player gets its input in the form of binary data. Like your computer, your CD player processes its input data, and then produces output. There are two big differences between your computer and the CD player, though:

1. Your CD player can't do anything with its input except turn it into music. The processing part of the process is hard coded. If you try to "play" your Dreamweaver CD, the CD player doesn't know what to do with the data.
2. Your computer can't turn your audio CD into sound like your CD player can. That's surprising, huh? Well, in fact, your computer doesn't really "know" how to do much of anything. It can load some data from a disc, and maybe do some rudimentary arithmetic, but certainly nothing as complex as playing music or even typing a letter. Compared to your CD player, your computer is just plain "dumb".

To do much of anything at all, your computer relies on programs. With the right program, you can play your audio CDs, as well as create Web pages, write your term paper, and browse the Internet. Without programs, your computer is just a paperweight.

Before Programs

- Early computers (like ENIAC) worked like my Discman
 - “Hardwired” to perform a particular calculation
 - Memory was used only to store data
- A mathematician came up with the idea of storing instructions in memory
 - Instructions were stored in memory
 - This was the first step towards a general-purpose machine

Slide 3

Before Programs

Duration: 00:01:32

Advance mode: Auto

Notes:

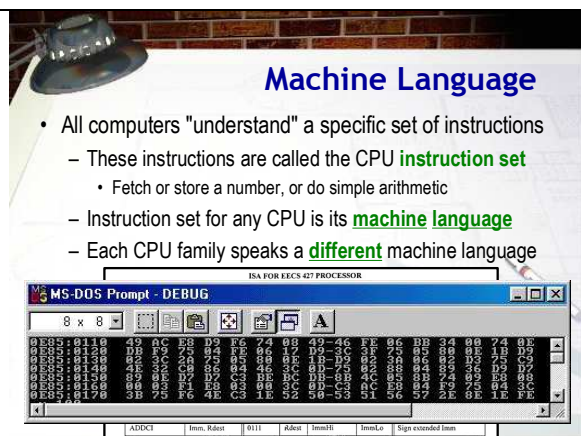
Today, we think of computers and computer programs as a single unit. This hasn't always been the case, though. Modern digital computers were developed in the late 1940s in Germany, England, and the US. These first computers had a processing unit, memory, and input/output devices, just like the computers we use today, but, surprisingly, they didn't have programs. Instead, these machines were "wired" to perform a specific function, just like the inexpensive 4-function calculator you can purchase at K-Mart, or, like my daughter's CD player.

Memory, in these early computers, was used only to store the data required for a calculation. To change the calculation performed on a computer like the Eniac, shown here, a team of several people worked for about a week "rewiring" the machine so that it carried out a new set of instructions.

The first real breakthrough in programming, (as we know it), came when a Princeton mathematician, [John von Neumann](#) realized that memory could be used to store computer

instructions along with data processed by the computer. This idea, called the stored program concept, means that you can use the same computer to play PacMan and to balance your checkbook—you simply change its program.

This stored program concept was an almost immediate success, and was the beginning of machine language. In recognition of this fact, we often say that modern computers use the von Neumann architecture.



Machine Language

- All computers "understand" a specific set of instructions
 - These instructions are called the CPU **instruction set**
 - Fetch or store a number, or do simple arithmetic
 - Instruction set for any CPU is its **machine language**
 - Each CPU family speaks a **different** machine language

ISA FOR EECS 427 PROCESSOR

MS-DOS Prompt - DEBUG

8 x 8

Address	Hex	ASCII
00000000	00 00 00 00	
00000001	00 00 00 00	
00000002	00 00 00 00	
00000003	00 00 00 00	
00000004	00 00 00 00	
00000005	00 00 00 00	
00000006	00 00 00 00	
00000007	00 00 00 00	
00000008	00 00 00 00	
00000009	00 00 00 00	
0000000A	00 00 00 00	
0000000B	00 00 00 00	
0000000C	00 00 00 00	
0000000D	00 00 00 00	
0000000E	00 00 00 00	
0000000F	00 00 00 00	
00000010	00 00 00 00	
00000011	00 00 00 00	
00000012	00 00 00 00	
00000013	00 00 00 00	
00000014	00 00 00 00	
00000015	00 00 00 00	
00000016	00 00 00 00	
00000017	00 00 00 00	
00000018	00 00 00 00	
00000019	00 00 00 00	
0000001A	00 00 00 00	
0000001B	00 00 00 00	
0000001C	00 00 00 00	
0000001D	00 00 00 00	
0000001E	00 00 00 00	
0000001F	00 00 00 00	
00000020	00 00 00 00	
00000021	00 00 00 00	
00000022	00 00 00 00	
00000023	00 00 00 00	
00000024	00 00 00 00	
00000025	00 00 00 00	
00000026	00 00 00 00	
00000027	00 00 00 00	
00000028	00 00 00 00	
00000029	00 00 00 00	
0000002A	00 00 00 00	
0000002B	00 00 00 00	
0000002C	00 00 00 00	
0000002D	00 00 00 00	
0000002E	00 00 00 00	
0000002F	00 00 00 00	
00000030	00 00 00 00	
00000031	00 00 00 00	
00000032	00 00 00 00	
00000033	00 00 00 00	
00000034	00 00 00 00	
00000035	00 00 00 00	
00000036	00 00 00 00	
00000037	00 00 00 00	
00000038	00 00 00 00	
00000039	00 00 00 00	
0000003A	00 00 00 00	
0000003B	00 00 00 00	
0000003C	00 00 00 00	
0000003D	00 00 00 00	
0000003E	00 00 00 00	
0000003F	00 00 00 00	
00000040	00 00 00 00	
00000041	00 00 00 00	
00000042	00 00 00 00	
00000043	00 00 00 00	
00000044	00 00 00 00	
00000045	00 00 00 00	
00000046	00 00 00 00	
00000047	00 00 00 00	
00000048	00 00 00 00	
00000049	00 00 00 00	
0000004A	00 00 00 00	
0000004B	00 00 00 00	
0000004C	00 00 00 00	
0000004D	00 00 00 00	
0000004E	00 00 00 00	
0000004F	00 00 00 00	
00000050	00 00 00 00	
00000051	00 00 00 00	
00000052	00 00 00 00	
00000053	00 00 00 00	
00000054	00 00 00 00	
00000055	00 00 00 00	
00000056	00 00 00 00	
00000057	00 00 00 00	
00000058	00 00 00 00	
00000059	00 00 00 00	
0000005A	00 00 00 00	
0000005B	00 00 00 00	
0000005C	00 00 00 00	
0000005D	00 00 00 00	
0000005E	00 00 00 00	
0000005F	00 00 00 00	
00000060	00 00 00 00	
00000061	00 00 00 00	
00000062	00 00 00 00	
00000063	00 00 00 00	
00000064	00 00 00 00	
00000065	00 00 00 00	
00000066	00 00 00 00	
00000067	00 00 00 00	
00000068	00 00 00 00	
00000069	00 00 00 00	
0000006A	00 00 00 00	
0000006B	00 00 00 00	
0000006C	00 00 00 00	
0000006D	00 00 00 00	
0000006E	00 00 00 00	
0000006F	00 00 00 00	
00000070	00 00 00 00	
00000071	00 00 00 00	
00000072	00 00 00 00	
00000073	00 00 00 00	
00000074	00 00 00 00	
00000075	00 00 00 00	
00000076	00 00 00 00	
00000077	00 00 00 00	
00000078	00 00 00 00	
00000079	00 00 00 00	
0000007A	00 00 00 00	
0000007B	00 00 00 00	
0000007C	00 00 00 00	
0000007D	00 00 00 00	
0000007E	00 00 00 00	
0000007F	00 00 00 00	
00000080	00 00 00 00	
00000081	00 00 00 00	
00000082	00 00 00 00	
00000083	00 00 00 00	
00000084	00 00 00 00	
00000085	00 00 00 00	
00000086	00 00 00 00	
00000087	00 00 00 00	
00000088	00 00 00 00	
00000089	00 00 00 00	
0000008A	00 00 00 00	
0000008B	00 00 00 00	
0000008C	00 00 00 00	
0000008D	00 00 00 00	
0000008E	00 00 00 00	
0000008F	00 00 00 00	
00000090	00 00 00 00	
00000091	00 00 00 00	
00000092	00 00 00 00	
00000093	00 00 00 00	
00000094	00 00 00 00	
00000095	00 00 00 00	
00000096	00 00 00 00	
00000097	00 00 00 00	
00000098	00 00 00 00	
00000099	00 00 00 00	
0000009A	00 00 00 00	
0000009B	00 00 00 00	
0000009C	00 00 00 00	
0000009D	00 00 00 00	
0000009E	00 00 00 00	
0000009F	00 00 00 00	
000000A0	00 00 00 00	
000000A1	00 00 00 00	
000000A2	00 00 00 00	
000000A3	00 00 00 00	
000000A4	00 00 00 00	
000000A5	00 00 00 00	
000000A6	00 00 00 00	
000000A7	00 00 00 00	
000000A8	00 00 00 00	
000000A9	00 00 00 00	
000000AA	00 00 00 00	
000000AB	00 00 00 00	
000000AC	00 00 00 00	
000000AD	00 00 00 00	
000000AE	00 00 00 00	
000000AF	00 00 00 00	
000000B0	00 00 00 00	
000000B1	00 00 00 00	
000000B2	00 00 00 00	
000000B3	00 00 00 00	
000000B4	00 00 00 00	
000000B5	00 00 00 00	
000000B6	00 00 00 00	
000000B7	00 00 00 00	
000000B8	00 00 00 00	
000000B9	00 00 00 00	
000000BA	00 00 00 00	
000000BB	00 00 00 00	
000000BC	00 00 00 00	
000000BD	00 00 00 00	
000000BE	00 00 00 00	
000000BF	00 00 00 00	
000000C0	00 00 00 00	
000000C1	00 00 00 00	
000000C2	00 00 00 00	
000000C3	00 00 00 00	
000000C4	00 00 00 00	
000000C5	00 00 00 00	
000000C6	00 00 00 00	
000000C7	00 00 00 00	
000000C8	00 00 00 00	
000000C9	00 00 00 00	
000000CA	00 00 00 00	
000000CB	00 00 00 00	
000000CC	00 00 00 00	
000000CD	00 00 00 00	
000000CE	00 00 00 00	
000000CF	00 00 00 00	
000000D0	00 00 00 00	
000000D1	00 00 00 00	
000000D2	00 00 00 00	
000000D3	00 00 00 00	
000000D4	00 00 00 00	
000000D5	00 00 00 00	
000000D6	00 00 00 00	
000000D7	00 00 00 00	
000000D8	00 00 00 00	
000000D9	00 00 00 00	
000000DA	00 00 00 00	
000000DB	00 00 00 00	
000000DC	00 00 00 00	
000000DD	00 00 00 00	
000000DE	00 00 00 00	
000000DF	00 00 00 00	
000000E0	00 00 00 00	
000000E1	00 00 00 00	
000000E2	00 00 00 00	
000000E3	00 00 00 00	
000000E4	00 00 00 00	
000000E5	00 00 00 00	
000000E6	00 00 00 00	
000000E7	00 00 00 00	
000000E8	00 00 00 00	
000000E9	00 00 00 00	
000000EA	00 00 00 00	
000000EB	00 00 00 00	
000000EC	00 00 00 00	
000000ED	00 00 00 00	
000000EE	00 00 00 00	
000000EF	00 00 00 00	
000000F0	00 00 00 00	
000000F1	00 00 00 00	
000000F2	00 00 00 00	
000000F3	00 00 00 00	
000000F4	00 00 00 00	
000000F5	00 00 00 00	
000000F6	00 00 00 00	
000000F7	00 00 00 00	
000000F8	00 00 00 00	
000000F9	00 00 00 00	
000000FA	00 00 00 00	
000000FB	00 00 00 00	
000000FC	00 00 00 00	
000000FD	00 00 00 00	
000000FE	00 00 00 00	
000000FF	00 00 00 00	

Slide 4

Machine Language

Duration: 00:01:47

Advance mode: Auto

Notes:

Every computer understands a particular, specific set of instructions, what we refer to as the CPU's instruction set. Here's the instruction set for an EECS 427 processor, for instance. The individual instructions for even the newest CPUs, are really pretty rudimentary; there are instructions for fetching a byte from memory, and for storing a byte back in memory, as well as instructions for adding, subtracting, and multiplying, etc.

Since the commands in the instruction set are the only thing that the computer understand, we call the instruction set for a particular computer its machine language. These machine languages are not universal; each one is specific to a particular CPU.

Here is an Intel Pentium machine language program. The rows of numbers on the left are the memory locations (called addresses) where each instruction is stored. The columns of numbers and letters on the right are the actual machine language instructions. The instructions stored in memory are actually binary numbers, but here they are displayed in hexadecimal (base 16) notation to make them easier to read.

As you can see by looking at the illustration, (and from your experiences using Debug in previous lectures), machine language is not very much like human language. Machine language is a numeric language, because the memory inside your computer can only store numeric data. Even when you work with text (such as viewing a web page), the computer is working with binary numbers. Because of this, writing machine language programs is very slow, tedious, and error-prone. Let's take a look at that process.

Machine Language

We need to calculate the total price. The sales tax rate is 8.65 %. Your program needs to multiply the purchase price by the tax rate, and then add the results and the price and store them in the total price field.

I need to know:

- What is the op-code to load from memory?
- Where is the purchase price stored in memory?
- What is the op-code to multiply?
- What do I multiply by?
- What is the op-code to add two values?
- What is the op-code to store a value in memory?

I need to:

- Load the purchase p
- Multiply by the sale
- Add result and purc
- Store final result in

Machine Language

Program Entered And Executed As Machine Language

Slide 5**Machine Language**

Duration: 00:01:48

Advance mode: Auto

Notes:

Suppose your boss asks you to write an application for a large department store. Part of the program requires you to calculate the total price of all purchases, including sales tax. To write this program in machine language, you'll have to mentally perform several translation steps.

The first step is to develop a set of unambiguous instructions or procedures that correctly calculate the price. These logical instructions are called an algorithm. It's important to perform this part carefully, because if you don't identify the correct steps and the correct order, your program will produce incorrect results, just as if you'd made a mistake in the steps required for a recipe.

A correct algorithm isn't sufficient to create a correct program, however. As a machine language programmer, you then need to translate that algorithm into the individual instructions that your CPU can perform, such as loading values from memory and adding, subtracting and multiplying.

After you've completed that task, though, you're still not finished. To tell the machine to load a value from memory, or to multiply two numbers, you'll need to look up the actual binary OP-code for each instruction, and enter that into memory. Thus to write a machine-language programmer, you actually perform three different kinds of translations:

- from the original program to an algorithm that solves the problem
- from the solution algorithm into the instruction-set of the CPU
- from the instruction-set to the actual binary OP-codes that the CPU can understand.

The Second Generation

- Machine language was difficult, but more efficient than building new "hard-wired" computing machines
- Larger programs became very hard to maintain
 - Programmers used "core dumps" to look at raw memory
- Why not put the computer to work doing this translation?

Slide 6**The Second Generation**

Duration: 00:01:34

Advance mode: Auto

Notes:

Despite the difficulties of using machine language, the number and size of the programs written expanded dramatically in the late 1940s and early 1950s. Machine-language programs were difficult to write, but creating them was a whole lot easier, and certainly less expensive, than building new "hard-wired" computing machines or employing thousands of "human computers" performing calculations the hard way.

These early "lean-and-mean" machine language programs quickly though became very large. Soon, they were too large to easily understand and maintain. If something goes wrong in one part of a machine language program, the programmer has to create a print-out showing the values in each memory cell when the error occurred—(this is called a "core dump"). That's what you see when you use the -d command in debug.

Core dump in hand, the programmer then must translate the values stored in memory into the basic instructions that the computer can perform: adding two numbers, perhaps, or

storing a value at a particular location. Only after the raw machine values are translated into their corresponding computer instructions, is the programmer ready to unravel the problem.

Programmers soon discovered that the computer itself could be put to work performing the painstaking but tedious task of translating memory values into the corresponding mnemonic "operation code" (or op code), so that one part of the debugging chore was lightened. That discovery lead to the next generation of ancient computer languages.

Assembly Language

We need the sales. Your program purchase then add store the

06	32	99	C6	06	34	99	FF	C6	06	35	99	00	EB	99	00
AC	E8	61	E2	74	38	3C	0D	74	34	3A	06	02	96	74	2E
3A	C3	74	2A	3C	3A	74	03	E9	5F	FF	80	3E	F0	97	02
75	05	E8	74	00	EB	D9	46	EB	14	E9	4D	FF	B0	D1	89
E9	CB	E5	BA	FB	8A	E9	C5	E5	4E	5F	9D	F9	C3	4E	EB
51	00	CF	01	81	CD	00	00	EB	12	E2	46	E8	E4	DF	74
0D	E8	45	00	AC	E8	41	00	81	CD	00	40	EB	34	3C	0D
75	09	00	00	AA	81	CD	00	40	EB	CE	E8	2B	00	E8	EC
DF	06	57	51	0E	07	BF	EF	8E	B9	06	00	81	CD	00	40
F2	AE	75	0B	81	E5	FF	BF	B8	01	00	D3	E0	0B	E8	59
5F	07	00	00	AA	5F	9D	F8	C3	AA	41	FE	06	34	99	C3
2E	C7	06	A1	90	00	00	2E	89	0E	2B	91	2E	89	26	2D
91	2E	89	36	2F	91	FC	2E	89	0E	94	90	2E	C7	06	96
90	00	00	2E	C7	06	A9	90	00	00	2E	06	9A	90	00	00
00	2E	C7	06	66	91	5B	5D	2E	C7	06	68	91	7C	3C	2E
C7	06	6A	91	3E	2B	2E	C7	06	6C	91	3D	3B	E8	83	09
73	13	B8	FF	FF	53	26	8B	1D	26	3A	0F	73	03	08	02
00	5B	E9	23	01	2E	89	36	A3	90	53	57	55	8D	1E	AB
90	2E	F6	06	A2	90	20	75	A3	AC	E8	0F	0A	72	3C	E8
77	09	74	37	E8	A4	09	75	18	2E	F6	06	A2	90	20	75
05	E8	3F	09	E8	26	2E	F6	06	A2	90	41	74	1E	4E	EB

Executed machine language

Slide 7

Assembly Language

Duration: 00:01:45

Advance mode: By user

Notes:

Remember that a program is just a set of instructions, stored in memory that the computer can obey. The computer, however, can only understand machine language, nothing else at all. That creates a big problem from our point of view, because writing machine language programs is tedious and error prone for humans beings. We have to store exactly the correct numeric instruction in exactly the correct memory location; if you are off by even one byte, your program will crash, and, if you're lucky, only lock up the computer.

Although meticulously typing numbers into memory is hard for us, it's not hard for the computer. This lead early computer designers to develop assembly language—a human readable form of machine language. Programmers still had to create an algorithm, but in the second step, assembly language programmers typed in short mnemonic instructions, like MOV AH, 3E, which tells the computer to store a value in one of the CPUs registers. Then, instead of looking up the actual machine language instruction, B43E, the programmer translates the mnemonic instruction into the actual machine language instruction, using a program called an assembler.

Both machine language and assembly language are low-level languages. This means that they are specific to a particular brand of CPU. A machine language or an assembly language program written for Intel's Pentium processor won't run at all on the Power PC CPU in a pre-intel Mac, or the Sparc inside a Sun workstation.

Libraries & Interpreters

- Libraries store common routines for later reuse
- Interpreters and Virtual Machine Languages
 - Speedcoding, Shortcode, Flowmatic
- Interpreters used today
 - HTML
 - Shell scripts
 - JavaScript

© 1996 Randy Glasbergen

Slide 8

Notes:

Computers follow their programmed instructions in a literal-minded, mechanical way. You can't just tell your computer to "print the budget report"; you have to explain every single tiny step. When you program in machine or assembly language, though, things seem even worse. Something as simple as printing a sentence on the screen can take half a page of code; and often, it's the same half page of code that you've written a dozen times already.

Can't the computer be put to work remembering all of those thousands of tiny details, so you, the programmer, don't have to? You bet; that's exactly what the early assembly-language programmers did. To lessen the burden of repetition, and to

Slide 9

Compilers

Duration: 00:01:60

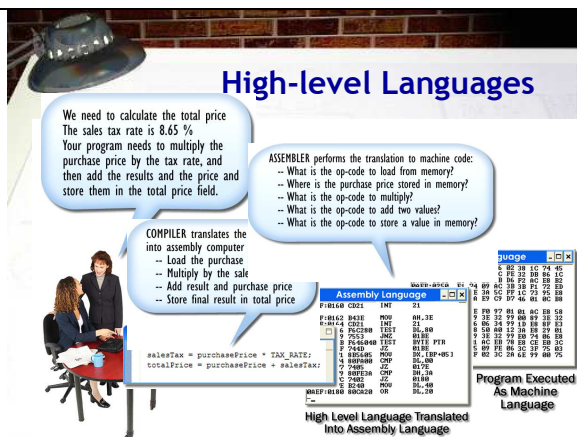
Advance mode: Auto

interpreter's time was spent translating from "virtual instructions" into machine language, interpreted programs ran much slower than hand written machine language programs. And, in those days, people were cheap, but computers were expensive.

A programmer named Grace Hopper, is credited with an insight that seems obvious in retrospect. Instead of translating Speedcode into machine code every time you run the program, just do the translation once. Save the translated code on disk or tape, and reuse it every time you need to run your program. This invention was called the compiler. Today, most programming languages use some form of compiler.

One way to understand the difference between an interpreter and a compiler is to think about the different ways we convert between human languages. The interpreter is kind of like the interpreters at the United Nations. The speaker's words are translated as they occur. A compiler is more like the translator of a book. The translator produces a new manuscript that is independent of the original.

Hopper's language, called Flowmatic, was the last, and greatest, of the 2nd generation languages. Hopper is also credited with discovering the first computer bug when she was a Navy Lieutenant working at Harvard in 1945. Even though this is kind of a tongue-in-cheek joke, the actual bug along with the log book is currently in the Naval Historical archives.



High-level Languages

We need to calculate the total price
The sales tax rate is 8.65 %
Your program needs to multiply the purchase price by the tax rate, and then add the results and the price and store them in the total price field.

ASSEMBLER performs the translation to machine code:
-- What is the op-code to load from memory?
-- Where is the purchase price stored in memory?
-- What is the op-code to multiply?
-- What is the op-code to add two values?
-- What is the op-code to store a value in memory?

COMPILER translates the into assembly computer
-- Load the purchase
-- Multiply by the sale
-- Add result and purchase price
-- Store final result in total price

High Level Language Translated Into Assembly Language

Program Executed As Machine Language

Slide 10

High-level Languages

Duration: 00:01:06

Advance mode: Auto

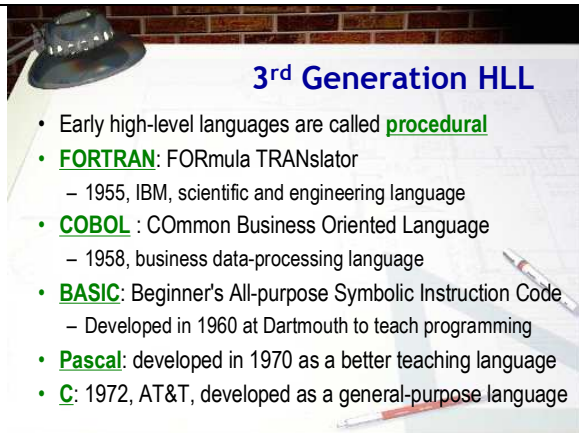
Notes:

Once people realized that computers could translate virtual assembly languages like Speedcode and Flowmatic into machine code, they began to wonder if, perhaps, computers could do the same for more "natural" languages. (Natural for human beings, that is.) This marked the beginning of the third generation of computer languages, called High Level Languages, or HLL.

The basic idea behind a high-level language is straightforward: instead of writing a computer program in terms that the computer uses, write it in terms of the problem to be solved like this:

Note that with high-level languages, the programmer only has to do one translation: from the problem to the logical steps needed to carry out the task. These steps, you'll recall, are called an algorithm, and for this reason, high-level languages are often called algorithmic languages.

Since different people want to solve different kinds of problems, different high-level languages were developed. Let's look quickly at the "big four".



3rd Generation HLL

- Early high-level languages are called **procedural**
- **FORTTRAN**: FORMula TRANslator
 - 1955, IBM, scientific and engineering language
- **COBOL**: COMmon Business Oriented Language
 - 1958, business data-processing language
- **BASIC**: Beginner's All-purpose Symbolic Instruction Code
 - Developed in 1960 at Dartmouth to teach programming
- **Pascal**: developed in 1970 as a better teaching language
- **C**: 1972, AT&T, developed as a general-purpose language

Slide 11

3rd Generation HLL

Duration: 00:02:21

Advance mode: Auto

Notes:

There are several different high-level languages that were developed in the 1950s to the early 1970s. These languages allow the programmer to write programs using English-like commands, and all of them are similar in one particular respect: they require the programmer to specify the precise actions that the computer should carry out. For that reason, these are known as procedural languages.

You don't have to recognize the commands used by each of these languages, but you should recognize the names and purposes behind these five:

FORTTRAN, which stands for FORMula TRANslator, was developed in 1955 at IBM, as a way to let scientists and engineers write their own programs. FORTRAN is still widely used in the scientific community, but not that much in the Computer Science world.

COBOL, or the COMmon Business Oriented Language, was released in 1958 by a consortium of Computer Manufacturers, along with the Department of Defense. COBOL was created to write accounting and business applications. Although there are versions of COBOL for personal computers, most COBOL programs run on mainframes, and handle "big jobs" like generating your utility bills each month.

BASIC, or the Beginner's All-purpose Symbolic linformation Code was developed in the early 1960s at Dartmouth College to teach students the fundamentals of computer programming, using the newly invented interactive time-sharing systems.

Ten years later, in 1970, Pascal was developed as a better language to teach Computer Science concepts, as well as good programming habits. For 20 years, almost all beginning programmers learned Pascal as their first "real" programming language.

In 1972, the C programming language was developed at AT&T, as part of the development of the Unix operating system, and to operate the telephone switching system infrastructure.

C is a relatively simple programming language that requires both discipline and sophistication to use safely and successfully. (A strange, but common saying, (that even ended up as a book title), is that "C gives you enough rope to shoot yourself in the foot." Today, almost all commercial software, such as Windows, Unix, Word, and Excel, is written in the C programming language.